AUTARQUIA DE ENSINO SUPERIOR DE ARCOVERDE

CENTRO DE ENSINO SUPERIOR DE ARCOVERDE

CURSO SUPERIOR EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RAIL Factory

Relatório Técnico

ERICK LEVI BASÍLIO DE AQUINO

ARCOVERDE - PE

2026

ERICK LEVI BASÍLIO DE AQUINO

RAIL Factory

Relatório Técnico apresentado como requisito parcial para a disciplina de Projeto de Desenvolvimento de Sistemas Web / Projeto de Desenvolvimento de Sistema

Corporativo, do curso superior em Análise em Desenvolvimento de Sistemas, sob orientação do Profº Dennys Cavalcanti Carvalho.

ARCOVERDE - PE

2026

HISTÓRICO DE REVISÃO

Data	Versão	Descrição
	1.0	Elaboração dos primeiros conteúdos para implementação no documento.

SUMÁRIO

1 DOCUMENTO DE DEFINIÇÃO DE ESCOPO (DDE)..... 6
1.1. INTRODUÇÃO..... 6
1.2. VISÃO GERAL DO DOCUMENTO..... 6
1.3. IDENTIFICAÇÃO DO PROJETO..... 6
1.4. OBJETIVOS DO PROJETO..... 6
1.5. JUSTIFICATIVA..... 7
1.6. IDENTIFICAÇÃO DOS REQUISITOS..... 7
1.6.1. Prioridades dos Requisitos..... 7
1.7. ESCOPO DO PRODUTO E ENTREGÁVEIS..... 8
1.7.1. Funcionalidades Previstas..... 8
1.7.2. Entregáveis..... 8
1.8. PREMISSAS E RESTRIÇÕES..... 8
1.8.1. Premissas..... 8
1.8.2. Restrições..... 8
1.9. CRITÉRIOS DE ACEITAÇÃO DO PROJETO..... 9
1.10. EXCLUSÕES DO ESCOPO..... 9
1.11. STAKEHOLDERS ENVOLVIDOS..... 9
1.12. RISCOS INICIAIS..... 9
2 DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS (ERS)..... 11
2.1. REQUISITOS FUNCIONAIS..... 11
2.2. REQUISITOS NÃO FUNCIONAIS..... 11
2.3. REGRAS DE NEGÓCIO..... 11
3 DOCUMENTO DE ESPECIFICAÇÃO DE MODELAGEM (DEM)..... 13
3.1 MODELAGEM DE DADOS..... 13
3.1.1 Entidade-Relacionamento..... 13
3.1.2 Dicionário de Dados..... 13
3.2 MODELAGEM COMPORTAMENTAL..... 13
3.2.1 Diagrama de Sequência..... 13
3.2.2 Diagrama de Estados..... 13
3.3. MODELAGEM ESTRUTURAL..... 13
3.3.1 Diagrama de Caso de Uso..... 13
3.3.2 Diagrama de Componentes..... 13
3.3.3 Diagrama de Arquitetura..... 13
3.4 MAPEAMENTO OBJETO-RELACIONAL (ORM)..... 13
3.5 BPMN (BUSINESS PROCESS MODEL AND NOTATION)..... 13
4 DOCUMENTO DE ESPECIFICAÇÃO DE INTERFACES (DEI)..... 14
4.1. WIREFRAMES..... 14
4.2. MOCKUPS..... 14

4.3. FLUXO DE NAVEGAÇÃO....14
5 DOCUMENTAÇÃO TÉCNICA....15
5.1. ARQUITETURA DO SISTEMA....15
5.1.1. Segmentação da Arquitetura....15
5.2. TECNOLOGIAS UTILIZADAS....15
5.2.1 Frontend....15
5.2.2. Backend....15
5.2.3. Banco de Dados....15
5.2.4. Ferramentas de Apoio....15
5.2.5. Padrões Adotados....15
5.2.6. Boas Práticas e Convenções....16
5.2.7. Requisitos de Infraestrutura....17
5.2.8. APIs e Integrações....17
5.2.9. Caracterização da API....17
5.3. REPOSITÓRIO E CÓDIGO-FONTE....17
6. MANUAL DO USUÁRIO....18
7. REFERÊNCIAS....20
8. APÊNDICE....21

1. DOCUMENTO DE DEFINIÇÃO DE ESCOPO (DDE)

1.1. INTRODUÇÃO

O RAIL Factory Manager é um software web de gestão fabril distribuída destinado a indústrias de manufatura que possuem mais de uma unidade produtiva. A solução atua como um painel central para a matriz e para as filiais, permitindo acompanhar produção, estoque e entregas de forma unificada, mas com dados isolados por fábrica.

A ideia central é oferecer uma plataforma única para registrar e consultar informações de produção, entrada de materiais, despacho de cargas, frota e horas trabalhadas, evitando planilhas dispersas, cadastros manuais de notas fiscais e falta de comunicação entre filiais e parceiros. Este documento apresenta o escopo inicial desse software e explica quais partes serão desenvolvidas nesta primeira entrega.

1.2. VISÃO GERAL DO DOCUMENTO

Este Documento de Definição de Escopo (DDE) apresenta o escopo detalhado do RAIL Factory Manager, uma plataforma web corporativa destinada à gestão fabril distribuída em indústrias de manufatura. Ele funciona como um “mapa” do arquivo, orientando o leitor sobre o que será desenvolvido e o que fica fora desta primeira etapa.

Neste DDE estão a identificação do projeto, os objetivos e a justificativa, o escopo do produto e os entregáveis, as funcionalidades previstas, as premissas e restrições, os critérios de aceitação, as exclusões de escopo, os stakeholders envolvidos e os riscos iniciais. O documento descreve o que o sistema terá nesta entrega de escopo, enquanto a especificação formal dos requisitos (ERS) será tratada em um documento separado em etapa posterior.

1.3. IDENTIFICAÇÃO DO PROJETO

O projeto identifica-se pelo nome RAIL Factory Manager e está vinculado ao Curso Superior de Análise e Desenvolvimento de Sistemas, 5º período. O autor e responsável técnico é Erick Levi Basílio de Aquino, sob orientação do professor Dennys Cavalcanti Carvalho. A proposta inicial foi submetida à instituição e validada como projeto corporativo, por atender aos critérios exigidos: integração bidirecional com sistemas legados e parceiros externos, arquitetura distribuída e automação de fluxos de dados entre produção, suprimentos,

logística e RH. O ambiente de desenvolvimento e homologação utiliza controle de versão (Git) e conteinerização (Docker), e o código-fonte e a documentação complementar ficam disponíveis nos repositórios e pastas do projeto.

1.4. OBJETIVOS DO PROJETO

Objetivo geral: Desenvolver um sistema web de gestão fabril distribuída para indústrias de manufatura com múltiplas unidades, oferecendo um painel unificado que isola os dados por fábrica e automatiza os fluxos de produção, entrada de materiais e despacho de cargas.

Objetivos específicos:

- Mapear, junto aos responsáveis de produção e logística, as necessidades de controle de ordens de produção, estoque por unidade fabril e acompanhamento de entregas.
- Projetar a arquitetura do sistema utilizando princípios de Domain-Driven Design (DDD) e separação por contextos (produção, inbound, outbound, frota, RH, mensageria e dashboard).
- Implementar um módulo de produção que permita cadastrar BOM (Bill of Materials), criar ordens de produção, registrar etapas e manter o estoque local por fábrica, com visão consolidada para a matriz.
- Implementar um módulo de inbound capaz de registrar a entrada de materiais, consumindo dados de notas fiscais eletrônicas (NF-e) via APIs da SEFAZ ou entrada manual, atualizando automaticamente o estoque.
- Implementar um módulo de outbound e logística que registre despachos, associe cargas a veículos e disponibilize status de entrega para clientes B2B por meio de APIs ou webhooks.
- Implementar um módulo de gestão de frota para cadastro de veículos, capacidade de carga e sugestão de rotas com base nas cargas planejadas.
- Implementar um painel (dashboard) com indicadores-chave de produção, estoque, logística, frota e horas trabalhadas, acessível via navegador em diferentes dispositivos.

1.5. JUSTIFICATIVA

As indústrias com múltiplas unidades fabris frequentemente mantêm controles de produção, estoque e logística em planilhas isoladas e sistemas que não se conversam, o que

provoca retrabalho, erros de digitação e atrasos na tomada de decisão. O cadastro manual de notas fiscais eletrônicas (NF-e) e a consolidação manual de estoques por unidade tornam difícil saber, em tempo hábil, quais produtos estão disponíveis e onde estão localizados.

O RAIL Factory Manager propõe uma solução centralizada para esse problema ao oferecer um sistema web que isola os dados por fábrica, mas permite à matriz enxergar o todo, automatizando o fluxo de entrada de materiais, o registro de produção e o despacho de cargas para clientes. Em vez de múltiplas planilhas e sistemas desconectados, a operação passa a contar com um painel único para registrar ordens, atualizações de estoque, entregas e horas trabalhadas, reduzindo erros manuais e melhorando a comunicação entre filiais e parceiros. Isso justifica o investimento no desenvolvimento do software em relação à manutenção do cenário atual baseado em controles dispersos.

1.6. IDENTIFICAÇÃO DOS REQUISITOS

Para facilitar a rastreabilidade, os requisitos do sistema serão identificados por um padrão que combina o nome da subseção onde estão descritos com um código numérico. Por convenção, os requisitos funcionais serão referenciados como [Nome da Subseção.RF-XX] e os requisitos não funcionais como [Nome da Subseção.NF-XX].

Por exemplo, um requisito funcional de cadastro de usuários poderá ser referenciado como [Cadastro de Usuários.RF-01], indicando que o requisito RF-01 está descrito na subseção “Cadastro de Usuários” dentro do bloco de requisitos funcionais. Da mesma forma, um requisito não funcional de disponibilidade poderá ser identificado como [Disponibilidade.NF-04], significando que o requisito NF-04 está localizado na seção “Requisitos Não Funcionais de Confiabilidade”, no bloco correspondente.

O presente DDE não detalha ainda a lista completa de requisitos; ele apenas define o padrão de identificação que será seguido quando o Documento de Especificação de Requisitos (ERS) for elaborado. Esse padrão garante que, ao citar um requisito em diagramas, testes ou na documentação técnica, seja possível localizar rapidamente sua descrição formal.

1.6.1. Prioridades Dos Requisitos

O que for implementado será priorizado em três níveis para orientar o planejamento e o MVP (mínimo produto viável). Essa priorização ajuda a tomar decisões quando o prazo ou os

recursos forem limitados: o essencial é inegociável; o importante é fortemente recomendado; o desejável pode ser adiado. A abordagem utilizada é a MoSCoW (Must, Should, Could, Would), amplamente usada em gestão de escopo (Cohn, 2010).

Prioridade	Descrição	Uso no projeto
Essencial	Obrigatório para o MVP. Sem isso, o sistema não atende ao objetivo.	Será implementado na versão inicial.
Importante	Impacta a qualidade e a experiência do usuário.	Fortemente recomendado no escopo inicial.
Desejável	Não compromete o núcleo da solução.	Pode ser adiado para versões futuras.

1.7. ESCOPO DO PRODUTO E ENTREGÁVEIS

O produto está vinculado a uma stack tecnológica planejada. No backend utilizam-se C# e .NET 8 (ASP.NET Core); no frontend, Blazor, mantendo a linguagem unificada e facilitando a manutenção. O armazenamento de dados é feito em PostgreSQL, com um banco dedicado por tenant (multi-tenancy por banco), e em Redis para cache e sessão. A mensageria é feita com RabbitMQ; a autenticação, com OAuth2 via Google. Completam a stack Entity Framework Core (acesso a dados), MassTransit (integração com RabbitMQ), xUnit (testes) e Docker/docker-compose (ambiente reproduzível). A arquitetura segue princípios de separação de camadas e de domínio bem definido, organizando o sistema em contextos como produção, inbound, outbound, frota, RH, mensageria e dashboard.

Como metas realistas para o ambiente acadêmico de homologação, espera-se que o sistema consiga responder requisições críticas (consulta de estoque, ordens de produção) em tempo aceitável para uso em laboratório (da ordem de poucos segundos em condições normais), suportar simultaneamente alguns tenants de teste (por exemplo, matriz e duas filiais) e realizar upload de arquivos de tamanho moderado (por exemplo, lotes de XML de NF-e usados nos cenários de teste), sem travamentos nem perda de dados.

1.7.1. Funcionalidades Previstas

No contexto de Identidade e Acesso haverá login com OAuth2 (Google), associação de usuário a um ou mais tenants, controle de acesso por funções (RBAC) — por exemplo Admin Matriz, Admin Filial, Operador — e auditoria rígida de alterações (quem alterou, quando, valor anterior). No motor de Produção haverá cadastro e gestão de BOM e ordens de produção, controle de etapas (status e conclusão), estoque local por tenant com movimentações de entrada e saída, visão de inventário global para a matriz e isolamento total de dados entre fábricas. No Supply Chain (Inbound) haverá consumo de APIs da SEFAZ e de fornecedores, atualização automática de estoque após validação e cadastro manual de entrada como fallback quando as APIs não estiverem disponíveis. Na Logística (Outbound) haverá registro de despacho, disparo de webhooks ao confirmar entrega (para que clientes B2B atualizem seus sistemas) e API de consulta de status para parceiros. Na Frota haverá cadastro de veículos, capacidade de carga (peso e volume), dimensões de produtos e sugestão de rotas. No RH haverá apontamento de horas por usuário e tenant e envio de dados para software contábil. Na Messageria haverá uso de RabbitMQ para eventos de domínio (entrada de material, despacho, alteração de ordem) e webhooks para sistemas externos. No Dashboard haverá indicadores de produção, estoque, logística, frota e horas, com interface responsiva e gráficos para acompanhamento gerencial.

1.7.2. Entregáveis

O projeto prevê quatro entregáveis principais. O primeiro é a plataforma web responsiva, com painéis distintos para Admin Matriz, Admin Filial e operadores, acessível em desktop, tablet e mobile, permitindo gestão e operação do dia a dia a partir do navegador. O segundo é o conjunto de bancos de dados: um PostgreSQL por tenant, armazenando usuários, BOM, ordens, estoque, entregas e logs, além de Redis para cache e sessão, com schema e dados isolados por fábrica. O terceiro é o manual do usuário, cobrindo o uso do painel, o trabalho com ordens, estoque e entregas, para que os clientes e operadores possam utilizar o sistema sem dependência exclusiva do suporte. O quarto é a documentação técnica, incluindo visão de arquitetura, documentação das APIs, aspectos de segurança e procedimentos de deploy, para manutenção e onboarding de novos desenvolvedores. Em paralelo, as entregas da disciplina estão definidas da seguinte forma: na Entrega 1 são entregues o DDE e o Diagrama de Casos de Uso; na Entrega 2, o Diagrama de Classes e o Diagrama de Sequência; na Entrega 3, o MVP com microserviços, frontend, CRUD e OAuth2 Google (o conteúdo do MVP — quais contextos e funcionalidades entram nessa entrega — segue a priorização

MoSCoW apresentada em aula, com foco no essencial: IAM, Produção básica e integrações iniciais); na Entrega 4, domínio, IAM e mensageria integrados, com teste de carga de upload em cenário controlado; na Entrega 5, a release final, a documentação técnica completa e o manual do usuário.

1.8. PREMISSAS E RESTRIÇÕES

1.8.1. Premissas

Para o planejamento do projeto, considera-se como verdadeiro o seguinte. Haverá acesso contínuo à internet e aos ambientes de desenvolvimento e deploy durante o ciclo do projeto. As APIs externas (SEFAZ, fornecedores, software contábil) estarão disponíveis dentro dos limites documentados pelos respectivos provedores; em caso de indisponibilidade temporária, o sistema prevê fallbacks manuais. O uso de OAuth2 com Google será permitido nos ambientes de desenvolvimento e homologação pela instituição (rede e políticas). Será possível definir pelo menos um tenant e perfis de usuário representativos (Admin, Operador, etc.) para testes e demonstração. A instituição permitirá o uso de ferramentas como Docker e Git. O MVP será priorizado com base no que for considerado essencial e importante para o objetivo do projeto, deixando o desejável para o backlog quando necessário; não se assume que todas as funcionalidades desejáveis estarão na primeira release.

1.8.2. Restrições

O projeto está sujeito ao prazo e ao cronograma de entregas definidos pela disciplina; não há flexibilidade para extrapolar as datas sem acordo formal com o orientador ou a instituição. O escopo limita-se à entrega de uma aplicação web responsiva; não está previsto o desenvolvimento de aplicativo mobile nativo nesta versão (interfaces futuras para mobile poderão consumir as mesmas APIs). A stack tecnológica é fixa: C#, .NET 8, Blazor, PostgreSQL, Redis, RabbitMQ e Docker, conforme aprovado na proposta; mudanças de tecnologia exigiriam revisão de escopo. As integrações com sistemas externos dependem da disponibilidade e da estabilidade das APIs de terceiros; em caso de indisponibilidade, os fallbacks manuais devem estar documentados e operáveis para não travar a operação. O prazo estimado para conclusão do MVP é de aproximadamente quatro meses, alinhado ao 5º período do curso.

1.9. CRITÉRIOS DE ACEITAÇÃO DO PROJETO

O projeto será considerado concluído e aceito quando forem atendidos os seguintes critérios. Do ponto de vista funcional, o administrador deve conseguir criar tenants, cadastrar BOM, ordens de produção e usuários e configurar permissões por papel e por tenant. O operador deve conseguir acessar o painel, visualizar ordens e estoque do seu tenant, registrar movimentações e acompanhar entregas. O gestor de Supply Chain (Inbound) deve conseguir consumir dados de APIs da SEFAZ e de fornecedores (ou simulação/mock) e registrar entrada manual de materiais quando as APIs não estiverem disponíveis; a atualização de estoque após entrada deve ser verificável nos painéis. O gestor de Frota deve conseguir cadastrar veículos, capacidade de carga (peso e volume), dimensões de produtos e obter sugestão de rotas coerente com as cargas planejadas. As APIs REST, os webhooks e o RabbitMQ devem estar operacionais e integrados nos cenários de teste definidos, com eventos sendo publicados e consumidos corretamente. O isolamento entre tenants deve ser garantido por meio do filtro obrigatório por tenant em todas as consultas; testes devem comprovar que um tenant não acessa dados de outro. A interface deve ter sido validada em desktop, tablet e mobile.

Do ponto de vista não funcional, as requisições críticas (como consulta de estoque e ordens de produção) devem responder em tempo satisfatório para uso em demonstração em laboratório, sem travamentos perceptíveis. Toda a comunicação deve utilizar TLS (HTTPS) e a autenticação deve ser feita com JWT, com tempo de expiração adequado. Deve existir auditoria rígida (registro de quem criou ou alterou e do valor anterior) nas entidades críticas: usuários, ordens de produção, estoque, entregas e configurações críticas. O sistema deve ser capaz de realizar upload e processamento de arquivos de entrada (por exemplo, lotes de XML de NF-e usados nos testes) sem falhas, dentro dos limites de infraestrutura disponíveis no ambiente acadêmico. Ao longo do período de homologação, o sistema deve manter-se estável durante as sessões de teste e demonstração, mesmo em caso de perda temporária de conexão com APIs externas (SEFAZ, fornecedores, contábil), desde que os fallbacks manuais previstos sejam utilizados.

1.10. EXCLUSÕES DO ESCOPO

Os itens abaixo ficam explicitamente fora do escopo inicial, para evitar dúvidas e expansão indevida (scope creep). Qualquer inclusão futura deve passar por revisão formal de escopo.

- Aplicativo mobile nativo. O escopo é aplicação web responsiva (desktop, tablet e mobile via navegador). APIs poderão ser consumidas por apps futuros, mas o desenvolvimento de app nativo não está previsto nesta versão.

- ERP contábil-financeiro completo. O sistema integra com software contábil existente (envio de horas, dados de RH). Não substitui e não implementa um ERP completo (contas a pagar/receber, conciliação bancária, DRE, etc.).

- Controle de maquinário industrial por IoT. O foco é gestão de produção, estoque, logística e frota em nível de sistema de informação. Controle direto de equipamentos no chão de fábrica (sensores, PLC, IoT) não está no escopo.

- Módulo completo de vendas e CRM. O sistema atende B2B e integra com sistemas do cliente (webhooks, APIs de status). Não inclui pipeline de vendas, funil comercial, gestão de leads ou CRM completo.

- Impressão nativa de etiquetas e documentos fiscais. Saída para impressão pode ser feita via integração ou exportação para outros sistemas; o RλIL-Factory não implementa drivers nem layout nativo de etiquetas e documentos fiscais.

1.11. STAKEHOLDERS ENVOLVIDOS

Os stakeholders do projeto são os atores que têm interesse ou participação direta no resultado. Eles são classificados em dois grupos: stakeholders de projeto (papel na condução e avaliação do projeto acadêmico) e stakeholders do produto (usuários do sistema ou parceiros que consomem suas interfaces). A tabela abaixo lista cada stakeholder com cargo (função ou papel formal), responsabilidade e, quando aplicável, a pessoa ou referência que ocupa o cargo.

Cargo	**Responsabilidade**	**Nome / Referência**
Administrador da Matriz	Gestão global do sistema, configuração de tenants (fábricas), visão consolidada de inventário e produção; acesso a dados de todas as unidades.	Usuário do sistema (perfil Admin Matriz).
Administrador da Filial	Gestão da unidade fabril, usuários locais e permissões restritas à sua fábrica; não acessa dados de outras filiais.	Usuário do sistema (perfil Admin Filial).
Operador de Produção	Uso do chão de fábrica: BOM, ordens de produção, estoque local, movimentações.	Usuário do sistema (perfil Operador).
Gestor de Supply Chain (Compras/Inbound)	Entrada de materiais, integração com SEFAZ e fornecedores, atualização de estoque.	Usuário do sistema.
Gestor de Logística e Distribuição	Despacho, integração com clientes B2B, status de entregas.	Usuário do sistema.
Gestor de Frota	Cadastro de veículos, rotas, capacidade de carga, sugestão de rotas.	Usuário do sistema.
Gestor de RH e Governança	Controle de acesso (RBAC), apontamento de horas, integração com software contábil.	Usuário do sistema.

Parceiro externo	Sistemas de clientes, fornecedores ou contábil que consomem APIs e webhooks do RλIL-Factory.	Sistemas externos (não é usuário do painel).
Professor orientador	Avaliação acadêmica, definição de escopo e prazos, acompanhamento do projeto. Não é usuário do sistema.	Prof. Dennys Cavalcanti Carvalho.
Autor do projeto	Responsabilidade técnica, desenvolvimento e entrega do sistema. Não é usuário do sistema.	Erick Levi Basílio de Aquino.

1.12. RISCOS INICIAIS

Foram identificados riscos iniciais que podem afetar o prazo, a qualidade ou a segurança do projeto. Para cada um deles foi definida uma estratégia de mitigação, de modo a reduzir a probabilidade ou o impacto. O acompanhamento desses riscos deve ser feito ao longo do ciclo de vida do projeto; novos riscos identificados durante a execução devem ser documentados e tratados da mesma forma. A tabela abaixo resume risco, impacto, probabilidade e mitigação.

Risco	**Impacto**	**Prob.**	**Mitigação**
Prazos apertados da disciplina	Atraso na implementação de partes do escopo	Alta	Focar primeiro nas funcionalidades essenciais definidas no DDE; deixar desejáveis para versões futuras.
Falta de familiaridade com tecnologias da stack (C#, Blazor, RabbitMQ)	Retrabalho, bugs, lentidão no desenvolvimento	Alta	Reservar tempo para estudo guiado (tutoriais e exemplos simples) antes de implementar módulos críticos; começar

			com um protótipo mínimo e evoluir gradualmente.
Conflitos de horário e carga de outras disciplinas	Menor tempo disponível para o projeto	Média	Planejar cronograma semanal com blocos dedicados ao projeto; reduzir escopo se necessário, priorizando o que será demonstrado na avaliação.
Indisponibilidade ou mudança nas APIs externas (SEFAZ, fornecedores, contábil)	Integrações quebradas ou impossíveis de testar em ambiente real	Média	Utilizar mocks e dados simulados para os cenários principais; documentar o comportamento esperado das integrações e manter fallback manual.
Limitações de infraestrutura (máquina local ou laboratório)	Lentidão, travamentos, dificuldade com múltiplos serviços	Média	Utilizar ambientes leves de desenvolvimento (docker-compose simplificado); evitar cenários irreais de carga; testar com poucos tenants e conjuntos pequenos de dados.